

RELATÓRIO DE ACOMPANHAMENTO DE REMEDIAÇÃO DE SEGURANÇA — REACT NATIVE

Aplicação: FUTAPP — Sistema de Reservas de Ginásios

Versão: React Native 0.81.5 / Expo SDK 54.0.x

Data: 11/06/2026

Auditor: Auditoria automatizada + revisão técnica de segurança

Escopo: OWASP Mobile Top 10 — M1 a M10

Tipo de relatório: Acompanhamento de correções aplicadas e pendências remanescentes

SUMÁRIO EXECUTIVO

Após o diagnóstico inicial de segurança, foram aplicadas **12 correções** no projeto React Native, reduzindo parcialmente a superfície de ataque relacionada a configurações inseguras, exposição de logs, validação de entrada, armazenamento inseguro em parâmetros de rota, cache HTTP e hardening básico de build.

Apesar dos avanços, o risco geral da aplicação ainda permanece **alto/crítico**, pois os principais pontos estruturais ainda dependem de backend real, autenticação server-side, JWT, criptografia, HTTPS, SecureStore, rate limiting, CORS restrito e controles de privacidade.

Correções aplicadas: 12

Pendências remanescentes: 13

Status geral: Remediação parcial concluída

Score residual de risco: ☒ Crítico ☐ Alto ☐ Médio ☐ Baixo

INDICADORES DE REMEDIAÇÃO

Itens corrigidos: 12

Itens parcialmente mitigados: 4

Itens pendentes por dependência de backend/infraestrutura: 13

Itens críticos ainda pendentes: Sim

Classificação atual do risco residual:

☒ Crítico ☐ Alto ☐ Médio ☐ Baixo

Motivo da classificação:

As correções aplicadas melhoraram a segurança do frontend e das configurações do projeto, porém vulnerabilidades centrais ainda permanecem ativas, principalmente autenticação feita no cliente, senhas em texto plano, ausência de JWT, ausência de HTTPS, ausência de SecureStore, falta de rate limiting e uso de `json-server` como backend.

CORREÇÕES APLICADAS

ID: FIX-001

Arquivo alterado: `.gitignore`

Vulnerabilidades relacionadas: VULN-M8-001, VULN-M9-001

Status: ☒ Corrigido ☐ Parcial ☐ Pendente

O que foi feito:

Foram adicionados `json-server/db.json` e arquivos `.env` ao `.gitignore`, impedindo que arquivos com dados sensíveis, credenciais, variáveis de ambiente e banco local sejam versionados novamente.

Risco mitigado:

Redução do risco de exposição de dados pessoais, senhas, variáveis de ambiente e informações sensíveis no repositório Git.

Impacto na segurança:

C: ☒ Alto ☐ Médio ☐ Baixo

I: ☐ Alto ☒ Médio ☐ Baixo

A: ☐ Alto ☐ Médio ☒ Baixo

Observação:

A correção impede novos commits acidentais, mas não remove dados sensíveis que já possam existir no histórico do Git.

Recomendação complementar:

Caso `db.json` ou `.env` já tenham sido commitados anteriormente, executar limpeza do histórico com BFG Repo Cleaner ou `git filter-repo`.

ID: FIX-002

Arquivo alterado: `app.json`

Vulnerabilidade relacionada: VULN-M8-002

Status: ☒ Corrigido ☐ Parcial ☐ Pendente

O que foi feito:

Foram adicionadas configurações de segurança no `app.json`, incluindo `allowBackup: false` para Android e `NSAllowsArbitraryLoads: false` para iOS.

Risco mitigado:

Redução do risco de extração de dados via backup Android e redução do risco de conexões inseguras arbitrárias no iOS.

Impacto na segurança:

C: ☒ Alto ☐ Médio ☐ Baixo

I: ☐ Alto ☒ Médio ☐ Baixo
A: ☐ Alto ☐ Médio ☒ Baixo

Observação:

Essa correção fortalece a configuração mobile, mas ainda não substitui a necessidade de HTTPS real, ATS completa e Network Security Config restritiva.

Recomendação complementar:

Adicionar também `usesCleartextTraffic: false` no Android e configurar ATS com domínio de produção quando a API HTTPS estiver disponível.

ID: FIX-003

Arquivo alterado: `package.json`

Vulnerabilidade relacionada: VULN-M2-002

Status: ☒ Corrigido ☐ Parcial ☐ Pendente

O que foi feito:

O pacote `json-server` foi movido de `dependencies` para `devDependencies` e sua versão foi pinada, removendo o uso de `^`.

Risco mitigado:

Redução do risco de o `json-server` ser tratado como dependência de produção e redução de builds não determinísticos causados por versionamento frouxo.

Impacto na segurança:

C: ☐ Alto ☒ Médio ☐ Baixo
I: ☐ Alto ☒ Médio ☐ Baixo
A: ☐ Alto ☒ Médio ☐ Baixo

Observação:

A correção reduz o risco de supply chain, mas o uso funcional do `json-server` como backend ainda permanece um risco arquitetural enquanto ele for usado para autenticação e persistência.

Recomendação complementar:

Substituir `json-server` por backend real antes de qualquer uso em produção.

ID: FIX-004

Arquivo criado: `metro.config.js`

Vulnerabilidades relacionadas: VULN-M6-002, VULN-M7-002

Status: ☒ Corrigido ☐ Parcial ☐ Pendente

O que foi feito:

Foi criado `metro.config.js` com `drop_console: true` para builds de produção.

Risco mitigado:

Redução da exposição de dados sensíveis em logs de produção, como e-mails, senhas, tokens, payloads de API e mensagens técnicas internas.

Impacto na segurança:

C: ☒ Alto ☐ Médio ☐ Baixo

I: ☐ Alto ☒ Médio ☐ Baixo

A: ☐ Alto ☐ Médio ☒ Baixo

Observação:

Essa correção mitiga logs em produção, mas não substitui o uso de um logger seguro e sanitizado no código.

Recomendação complementar:

Manter o logger sanitizado e garantir que builds de produção estejam usando corretamente a configuração do Metro.

ID: FIX-005

Arquivo criado: `eas.json`

Vulnerabilidades relacionadas: VULN-M7-001, VULN-M8-002

Status: ☒ Corrigido ☐ Parcial ☐ Pendente

O que foi feito:

Foram criados perfis separados de build: `development`, `preview` e `production`.

Risco mitigado:

Redução do risco de gerar builds de produção com configurações de desenvolvimento, debug ativo, logs expostos ou ambiente incorreto.

Impacto na segurança:

C: ☐ Alto ☒ Médio ☐ Baixo

I: ☒ Alto ☐ Médio ☐ Baixo

A: ☐ Alto ☒ Médio ☐ Baixo

Observação:

A separação de perfis é uma base importante para hardening, mas ainda não implementa por si só root detection, anti-tampering, code signing ou obfuscação completa.

Recomendação complementar:

Configurar variáveis por ambiente, OTA code signing, runtime version e políticas de produção no EAS.

ID: FIX-006

Arquivo criado: `src/utils/logger.ts`

Vulnerabilidade relacionada: VULN-M6-002

Status: ☒ Corrigido ☐ Parcial ☐ Pendente

O que foi feito:

Foi criado um logger centralizado com guarda `__DEV__` e sanitização de campos sensíveis, como senha, token, e-mail e outros dados pessoais.

Risco mitigado:

Redução do vazamento de dados sensíveis em logs locais, logs de desenvolvimento e possíveis ferramentas futuras de crash reporting.

Impacto na segurança:

C: ☒ Alto ☐ Médio ☐ Baixo

I: ☐ Alto ☐ Médio ☒ Baixo

A: ☐ Alto ☐ Médio ☒ Baixo

Observação:

A correção é efetiva desde que todos os `console.log`, `console.warn` e `console.error` sensíveis sejam substituídos pelo logger.

Recomendação complementar:

Adicionar revisão automatizada ou regra de lint para bloquear `console.log` em arquivos de produção.

ID: FIX-007

Arquivo alterado: `src/apiService/api.ts`

Vulnerabilidade relacionada: VULN-M9-002

Status: ☒ Corrigido ☐ Parcial ☐ Pendente

O que foi feito:

Foram adicionados headers de cache, incluindo `Cache-Control: no-store`, sanitização de erros em produção e a função `getReservaPorId()` para buscar uma reserva diretamente pelo ID.

Risco mitigado:

Redução do risco de cachear respostas com dados pessoais e financeiros. Também reduz exposição de mensagens técnicas ao usuário final.

Impacto na segurança:

C: ☒ Alto ☐ Médio ☐ Baixo

I: ☐ Alto ☒ Médio ☐ Baixo

A: ☐ Alto ☐ Médio ☒ Baixo

Observação:

A função `getReservaPorId()` também apoia a correção da vulnerabilidade de JSON completo em parâmetros de rota.

Recomendação complementar:

Quando houver JWT, incluir automaticamente o header `Authorization: Bearer <token>` em todas as chamadas autenticadas.

ID: FIX-008

Arquivo alterado: `src/app/login.tsx`

Vulnerabilidades relacionadas: VULN-M4-002, VULN-M6-002

Status: ☒ Corrigido ☐ Parcial ☐ Pendente

O que foi feito:

Foi adicionada validação de e-mail por regex, substituição de `console.log` por logger seguro e mensagem de erro genérica para o usuário.

Risco mitigado:

Redução de entradas inválidas no formulário de login e redução de exposição de detalhes técnicos ou dados sensíveis em logs e mensagens de erro.

Impacto na segurança:

C: ☐ Alto ☒ Médio ☐ Baixo

I: ☐ Alto ☒ Médio ☐ Baixo

A: ☐ Alto ☐ Médio ☒ Baixo

Observação:

A validação client-side melhora a UX e reduz entradas inválidas, mas não substitui validação server-side.

Recomendação complementar:

Quando o backend real for criado, validar e-mail e senha também no endpoint `POST /auth/login`.

ID: FIX-009

Arquivo alterado: `src/app/cadastro.tsx`

Vulnerabilidades relacionadas: VULN-M4-002, VULN-M4-003

Status: ☒ Corrigido ☐ Parcial ☐ Pendente

O que foi feito:

Foi adicionada validação de e-mail, bloqueio real para senhas com menos de 8 caracteres e substituição de `console.log` por logger seguro.

Risco mitigado:

Redução do risco de cadastro com senhas fracas e redução da exposição de dados em logs.

Impacto na segurança:

C: ☐ Alto ☒ Médio ☐ Baixo

I: ☐ Alto ☒ Médio ☐ Baixo

A: ☐ Alto ☐ Médio ☒ Baixo

Observação:

A política de senha deixou de ser apenas visual e passou a ser aplicada no fluxo de cadastro.

Recomendação complementar:

Adicionar política mais robusta no backend, com validação de força, bloqueio de senhas comuns e hashing seguro.

ID: FIX-010

Arquivo alterado: `src/app/lista.tsx`

Vulnerabilidades relacionadas: VULN-M4-001, VULN-M6-002

Status: ☒ Corrigido ☐ Parcial ☐ Pendente

O que foi feito:

Foi removido o envio de `JSON.stringify(item)` nos parâmetros de rota. Também foram substituídas 3 ocorrências de `console.log` por logger seguro.

Risco mitigado:

Redução do risco de manipulação de objetos de reserva via URL, redução de exposição de dados financeiros e redução de logs sensíveis.

Impacto na segurança:

C: ☒ Alto ☐ Médio ☐ Baixo

I: ☒ Alto ☐ Médio ☐ Baixo

A: ☐ Alto ☒ Médio ☐ Baixo

Observação:

Essa foi uma das correções mais relevantes no frontend, pois removeu um vetor direto de manipulação de estado e autorização.

Recomendação complementar:

Validar no backend se o usuário autenticado possui permissão para acessar ou alterar a reserva solicitada.

ID: FIX-011

Arquivo alterado: `src/app/detalhar.tsx`

Vulnerabilidades relacionadas: VULN-M4-001, VULN-M3-002, VULN-M6-002

Status: ☒ Corrigido ☐ Parcial ☐ Pendente

O que foi feito:

A tela foi reescrita para buscar a reserva pelo ID via `GET /reservas/:id`, em vez de parsear JSON vindo da URL. Também foram substituídas 8 ocorrências de `console.log` por logger seguro.

Risco mitigado:

Redução do risco de object injection, manipulação de reserva via deep link, crash por payload malformado e vazamento de dados em logs.

Impacto na segurança:

C: ☒ Alto ☐ Médio ☐ Baixo

I: ☒ Alto ☐ Médio ☐ Baixo
A: ☐ Alto ☒ Médio ☐ Baixo

Observação:

A correção elimina o uso de JSON serializado na rota, mas a autorização ainda depende de backend seguro para ser plenamente confiável.

Recomendação complementar:

No backend real, validar se o `usuarioId` do token tem permissão para acessar, editar, aprovar ou rejeitar dados daquela reserva.

ID: FIX-012

Arquivo alterado: `src/app/home.tsx`

Vulnerabilidade relacionada: VULN-M3-003

Status: ☒ Corrigido ☐ Parcial ☐ Pendente

O que foi feito:

O logout foi alterado de `router.push("/login")` para `router.replace("/login")`, reduzindo o risco de retorno para telas autenticadas pelo histórico de navegação.

Risco mitigado:

Redução do risco de acesso indevido após logout usando o botão voltar do Android ou histórico interno da navegação.

Impacto na segurança:

C: ☐ Alto ☒ Médio ☐ Baixo
I: ☐ Alto ☒ Médio ☐ Baixo
A: ☐ Alto ☐ Médio ☒ Baixo

Observação:

A correção melhora o comportamento do logout, mas ainda não existe sessão real para invalidar.

Recomendação complementar:

Quando JWT e SecureStore forem implementados, o logout deve apagar tokens, limpar contexto de autenticação e invalidar refresh token no servidor.

VULNERABILIDADES PENDENTES

ID: PEND-001

Vulnerabilidade: VULN-M1-001 — Senhas em plaintext / bcrypt ou Argon2

Status: ☒ Pendente

Severidade residual: Crítica

Motivo de não ter sido corrigido:

A correção depende da substituição do `json-server` por um backend real com banco de dados e camada de autenticação.

Risco atual:

Senhas ainda podem permanecer armazenadas em texto plano enquanto a arquitetura atual for mantida.

Ação necessária:

Implementar backend com `POST /auth/login`, armazenar `senhaHash` e aplicar Argon2id ou bcrypt.

Responsável sugerido: Backend

Prazo sugerido: Imediato

ID: PEND-002

Vulnerabilidade: VULN-M3-001 — Autenticação server-side + JWT

Status: [X] Pendente

Severidade residual: Crítica

Motivo de não ter sido corrigido:

Requer criação de endpoint `POST /auth/login` no servidor.

Risco atual:

A autenticação continua dependendo do fluxo atual baseado em dados de usuário vindos da API e lógica no cliente.

Ação necessária:

Criar autenticação server-side, retornar access token, refresh token e remover qualquer retorno de senha para o app.

Responsável sugerido: Backend

Prazo sugerido: Imediato

ID: PEND-003

Vulnerabilidade: VULN-M3-002 — Sessão em URL params / SecureStore + AuthContext

Status: [X] Pendente parcial

Severidade residual: Crítica

Motivo de não ter sido corrigido:

A remoção do JSON da URL foi feita, mas a sessão segura depende da existência de JWT ou token real.

Risco atual:

Enquanto não houver token, AuthContext e SecureStore, a identidade do usuário ainda não possui prova criptográfica.

Ação necessária:

Implementar `AuthContext`, `expo-secure-store`, token JWT e proteção de rotas.

Responsável sugerido: Mobile / Backend

Prazo sugerido: Imediato

ID: PEND-004

Vulnerabilidade: VULN-M3-003 — Rate limiting no login

Status: [X] Pendente

Severidade residual: Alta

Motivo de não ter sido corrigido:

Requer middleware no servidor.

Risco atual:

Tentativas de login podem ser feitas sem bloqueio real no backend.

Ação necessária:

Implementar `express-rate-limit`, bloqueio progressivo, logs de tentativa e mensagens genéricas.

Responsável sugerido: Backend

Prazo sugerido: Curto prazo

ID: PEND-005

Vulnerabilidade: VULN-M5-001 — HTTPS / TLS

Status: [X] Pendente

Severidade residual: Crítica

Motivo de não ter sido corrigido:

Requer deploy da API em domínio real com certificado TLS.

Risco atual:

Tráfego HTTP continua inseguro caso usado fora do ambiente local.

Ação necessária:

Publicar API em HTTPS, configurar TLS 1.2+ ou TLS 1.3 e bloquear HTTP em produção.

Responsável sugerido: DevOps / Backend

Prazo sugerido: Imediato antes de deploy

ID: PEND-006

Vulnerabilidade: VULN-M5-002 — Certificate pinning

Status: [X] Pendente

Severidade residual: Alta

Motivo de não ter sido corrigido:

Requer servidor HTTPS real e configuração nativa no app.

Risco atual:

Mesmo com HTTPS futuro, o app ainda poderá aceitar certificados válidos de qualquer CA confiável se não houver pinning.

Ação necessária:

Configurar pinning de certificado ou public key pinning para o domínio da API.

Responsável sugerido: Mobile / DevOps

Prazo sugerido: Após HTTPS

ID: PEND-007

Vulnerabilidade: VULN-M6-001 — Consentimento LGPD

Status: [X] Pendente

Severidade residual: Alta

Motivo de não ter sido corrigido:

Requer tela de consentimento, banco de consentimentos e política de privacidade.

Risco atual:

O app ainda não possui fluxo completo de transparência, consentimento, revogação e direitos do titular.

Ação necessária:

Criar tela de consentimento, central de privacidade, política de privacidade, exportação de dados e exclusão de conta.

Responsável sugerido: Produto / Jurídico / Backend / Mobile

Prazo sugerido: Curto prazo

ID: PEND-008

Vulnerabilidade: VULN-M7-001 — Root/jailbreak/emulator detection

Status: [X] Pendente

Severidade residual: Alta

Motivo de não ter sido corrigido:

Requer biblioteca como `react-native-jailmonkey` ou equivalente.

Risco atual:

O app ainda pode ser executado livremente em dispositivos rootados, jailbroken, emuladores ou ambientes de análise dinâmica.

Ação necessária:

Implementar detecção de root, jailbreak, emulador, hook, debugger e resposta segura em produção.

Responsável sugerido: Mobile

Prazo sugerido: Médio prazo

ID: PEND-009

Vulnerabilidade: VULN-M8-001 — CORS aberto + ausência de rate limit no json-server

Status: [X] Pendente

Severidade residual: Crítica

Motivo de não ter sido corrigido:

Requer backend real com `helmet`, CORS restrito e `express-rate-limit`.

Risco atual:

Enquanto `json-server` for usado como API, permanece o risco de CRUD aberto, CORS permissivo e ausência de controle de abuso.

Ação necessária:

Substituir `json-server` por API real, restringir origens, proteger rotas e limitar requisições.

Responsável sugerido: Backend / DevOps

Prazo sugerido: Imediato

ID: PEND-010

Vulnerabilidade: VULN-M9-001 — SecureStore para token de sessão

Status: [X] Pendente

Severidade residual: Crítica

Motivo de não ter sido corrigido:

Depende da existência de token JWT ou token de sessão.

Risco atual:

Ainda não há armazenamento seguro de sessão no Keychain/Keystore.

Ação necessária:

Instalar com `npx expo install expo-secure-store` e armazenar access token/refresh token de forma segura.

Responsável sugerido: Mobile

Prazo sugerido: Após criação do JWT

ID: PEND-011

Vulnerabilidade: VULN-M9-002 — FLAG_SECURE / blur em background

Status: [X] Pendente parcial

Severidade residual: Alta

Motivo de não ter sido corrigido:

Requer plugin nativo ou implementação com `AppState` + `expo-blur`.

Risco atual:

Telas com dados sensíveis ainda podem aparecer em screenshots, gravações de tela ou snapshots do multitasking.

Ação necessária:

Aplicar blur ao enviar o app para background e avaliar uso de `FLAG_SECURE` em telas sensíveis.

Responsável sugerido: Mobile

Prazo sugerido: Curto prazo

ID: PEND-012

Vulnerabilidade: VULN-M2-001 — CVE crítico `shell-quote` CVSS 9.8

Status: [X] Pendente

Severidade residual: Crítica

Motivo de não ter sido corrigido:

Requer executar `npm audit fix` com cautela, pois pode causar alterações transitivas ou breaking changes.

Risco atual:

Permanece vulnerabilidade crítica na cadeia de dependências.

Ação necessária:

Executar `npm audit fix`, revisar lockfile, rodar testes e validar build Android/iOS/Web.

Responsável sugerido: DevOps / Mobile

Prazo sugerido: Imediato

ID: PEND-013

Vulnerabilidade: VULN-M10-001 — Ausência de criptografia real

Status: [X] Pendente

Severidade residual: Crítica

Motivo de não ter sido corrigido:

Depende das correções estruturais de backend, JWT, TLS, SecureStore e hashing de senha.

Risco atual:

O app ainda não possui criptografia adequada para senhas, sessão, transporte e armazenamento seguro.

Ação necessária:

Implementar Argon2id/bcrypt, JWT RS256/ES256, TLS, SecureStore, geração segura de tokens e rotação de chaves.

Responsável sugerido: Backend / Mobile / DevOps

Prazo sugerido: Imediato e contínuo

MATRIZ DE STATUS ATUAL

Status	Quantidade	Observação
Corrigido	12	Correções aplicadas principalmente no frontend/ configuração
Parcialmente mitigado	4	Melhorias feitas, mas dependem de backend/token para encerrar risco
Pendente	13	Requer backend real, infraestrutura, HTTPS, JWT ou libs nativas
Crítico residual	Sim	Autenticação, senha, HTTPS, CORS, JWT e criptografia ainda pendentes

RISCO RESIDUAL

Mesmo após as 12 correções, o risco residual permanece elevado porque as principais vulnerabilidades são estruturais.

Principais riscos ainda ativos

1. Senhas ainda dependem de backend real com hash seguro.
2. Autenticação ainda não é server-side.
3. Não há JWT nem token de sessão.
4. Não há SecureStore para sessão.
5. Ainda não há HTTPS real em produção.
6. Não há CORS restrito nem rate limiting em backend real.
7. Ainda há dependência crítica vulnerável a corrigir com `npm audit fix`.
8. Controles LGPD ainda não foram implementados.
9. Proteções contra engenharia reversa ainda não foram implementadas.
10. Criptografia real ainda depende da nova arquitetura.

PLANO DE REMEDIAÇÃO ATUALIZADO

Prioridade 1 — Imediato

1. Criar backend real e substituir `json-server`.
 2. Implementar `POST /auth/login`.
 3. Aplicar hash de senha com Argon2id ou bcrypt.
 4. Implementar JWT com expiração.
 5. Instalar e configurar `expo-secure-store`.
 6. Corrigir CVE crítico com `npm audit fix`.
 7. Configurar HTTPS em ambiente de homologação/produção.
 8. Implementar CORS restrito e rate limiting.
-

Prioridade 2 — Curto prazo

1. Criar `AuthContext` e proteger rotas.
 2. Remover totalmente dependência de `usuarioId` em URL como identidade.
 3. Implementar autorização server-side para reservas, participantes e solicitações.
 4. Criar fluxo de consentimento LGPD.
 5. Criar política de privacidade e central de privacidade.
 6. Configurar headers de segurança com `helmet`.
 7. Adicionar validação server-side com Zod/Yup/Joi.
-

Prioridade 3 — Médio prazo

1. Implementar certificate pinning.
 2. Implementar proteção contra screenshots ou blur em background.
 3. Implementar root/jailbreak/emulator/debug detection.
 4. Configurar OTA Updates com code signing.
 5. Adicionar pipeline CI/CD com SAST, secret scanning e dependency audit.
 6. Implementar rotação de chaves e gestão de secrets.
 7. Auditar novamente Android/iOS após build production.
-

CONCLUSÃO

As correções realizadas representam uma evolução importante na segurança do projeto, principalmente em relação a logs, configuração de build, `.gitignore`, validação básica de formulários, remoção de JSON sensível em parâmetros de rota e controle de cache.

No entanto, a aplicação ainda não deve ser considerada pronta para produção com dados reais, pois os riscos mais críticos continuam associados à arquitetura atual: uso de `json-server`, ausência de autenticação server-side, ausência de JWT, senhas sem hashing, ausência de HTTPS real, ausência de SecureStore e falta de controles LGPD.

A próxima etapa recomendada é iniciar a refatoração do backend e da autenticação, pois a maior parte das pendências críticas depende dessa base para ser concluída corretamente.

Re-auditoria recomendada:

Após implementação do backend real, JWT, HTTPS, SecureStore e correção das dependências críticas.